Representation and Preprocessing in Grammar-Constrained Decoding: Stack Indexes and Online Completion Lower Bounds

ZIJIAN ZENG*, UCSI University, Malaysia
JIAYIN CAO, Ningbo University, China
WEINAN DAI, Zhejiang University, China
RUI WANG, Nanchang Institute of Technology, China

Grammar-constrained decoding asks which model tokens preserve the existence of a valid completion. Tokens can cross lexical boundaries and trigger long parser reduction chains. We study this online problem with separate parameters for the language presentation, vocabulary, and streamed text.

Our main lower bound already holds for right-linear grammars and six one-byte tokens. Under the Online Matrix–Vector Multiplication conjecture, polynomial preprocessing cannot provide amortized $O(n^{1/2-\epsilon})$ time per step on one stream. The constructed matrix-language family admits an $O(1 + \sqrt{n}/w)$ algorithm. With shared preprocessing and independently initialized streams, the corresponding forbidden exponent is $1 - \epsilon$ because the horizon of each stream is shorter.

For deterministic pushdown presentations, we give an incremental regular-stack index and explicit compilation bounds. A classifier with s NFA states yields an index of size $2^{O(s)}$ with $O(\lceil M/w \rceil)$ mask materialization time; a block-tabulation variant exposes a tunable space–query–update tradeoff. Committed parser work is charged separately. Thus the relevant complexity parameters include representation, preprocessing budget, and update work, in addition to the language class. A typed-completion corollary delineates a semantic boundary using classical inhabitation complexity. Executable finite checks accompany the proofs.

*Corresponding author.

Authors' Contact Information: Zijian Zeng, UCSI University, Malaysia, 1002266693@ucsiuniversity.edu.my; Jiayin Cao, Ningbo University, China, 2795820153@qq.com; Weinan Dai, Zhejiang University, China, 21725058@zju.edu.cn; Rui Wang, Nanchang Institute of Technology, China, 2002007@nut.edu.cn.

1 Introduction

Structured language-model output is often consumed immediately by a schema validator, query engine, or compiler. Before sampling each token, a grammar-constrained decoder computes a vocabulary mask that preserves the possibility of producing an accepted output. This is an existential completion question. A token may span several lexemes, end inside a lexeme, or trigger many reductions before the parser consumes its next byte. The online cost therefore combines token alignment, configuration classification, and parser updates.

Our central question is how presentation and preprocessing affect exact token-completion masks. We give a conditional barrier for a succinct regular-language family and a stack-index construction for supplied deterministic pushdown presentations. These results use different preprocessing budgets, which we state explicitly. The lower bound is exponent-tight on its constructed family; it is not a tight characterization of general grammar-constrained decoding.

A two-row example. Let $A = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right)$ and $v = (1, 0)^{\mathsf{T}}$. Our grammar accepts universal blocks of the form $v\#aa!$, keeping their prefixes live. A separate branch permits a terminal query byte ? after $v\#a^i$ exactly when $(Av)_i = 1$. Thus ? is legal after `10#a` and illegal after `10#aa`. The actual stream commits both a bytes and then !, never the tested ?; a zero answer cannot invalidate subsequent row queries. Generalizing this construction gives the lower bound.

Results and technical ideas. For an $m \times m$ matrix, a right-linear grammar of size $\Theta(m^2)$ supports m online vectors using $\Theta(m^2)$ appended bytes in total. Each row answer is one bit of a six-token mask. Under OMv, polynomial preprocessing excludes amortized $O(n^{1/2-\epsilon})$ steps on this trace. For m independently initialized streams, each of length $\Theta(m)$, it excludes $O(n^{1-\epsilon})$ steps. A packed-row algorithm takes $O(1 + m/w)$ per step on the same family.

The upper bound starts from regular languages of pushdown configurations. After reversal, one bottom-to-top NFA simultaneously classifies the current stack for all candidate tokens. A fully determinized index stores its state at every persistent stack node: push computes a transition, and pop restores the parent annotation. Mask output takes $O(\lceil M/w \rceil)$ word writes. We bound the classifier size polynomially in the explicit pushdown and vocabulary inputs, expose the exponential determinization cost, and give a block-tabulation alternative with a selectable static budget. Parser reductions performed while committing the selected token remain explicit update work.

Relation to prior work. DOMINO addresses subword alignment [2]; XGrammar uses local mask caching, context-dependent checks, and persistent stacks [5]. Outlines indexes vocabulary transitions [21]; SynCode develops formal syntactic guarantees [20]; llguidance combines Earley parsing, lexing, and trie traversal [14]. Lexer/parser products [17] and GPU-oriented token precompilation [11] further develop this systems design space. Our bounds account for the operations represented by those techniques rather than compare model throughput.

The closest upper-bound predecessor is Parser Stack Classification (PSC) [13], whose regular classifier is executed by an $O(h)$ stack scan in Algorithm 2. VIABLE maintains reverse classifier annotations incrementally. Regularity, reversal, subset construction, and block tabulation are established automata tools; the result here is their explicit query/update/compilation accounting for exact completion masks.

Its significance is the separation of query latency from committed parsing, rather than a new regularity theorem.

On the lower-bound side, Bringmann, Grønlund, Künnemann, and Larsen formulate the NFA Acceptance hypothesis and show that it implies OMv [4]. Their fixed-word acceptance and colored-walk formulations provide important complexity context. Gibney and Thankachan study preprocessing a text for subsequent regular-expression queries [9]. Our static object is the grammar, the online object is appended text, and the answer is a candidate-completion bit along prefixes kept live by a universal branch. These distinctions specify the transfer that the proof must establish; regular-language OMv hardness alone is already part of the surrounding literature.

Online CFL recognition has a different predicate: it reports membership of each observed prefix. Dudek and Gawrychowski give an upper bound using OMv methods [6]. Earlier online Turing-machine lower bounds [7], sliding-window bounds with deletion [8], and offline parsing barriers [1, 12] concern different machines, updates, or outputs. The reduction below fixes these choices directly. Type-constrained generation [15] and semantic realizability [16] motivate the auxiliary typed-completion result in Section A.

2 Model and Results

Fix a byte alphabet Σ. The static input contains a CFG G, a DFA for each terminal's nonempty regular lexeme language, and a specified boundary/priority policy. Relational segmentation gives the regular substitution of these lexeme languages into $L(G)$. A deterministic implementation instead supplies a compatible finite-state streaming lexer and a normalized pushdown presentation P of the resulting byte language K. A finite DFA collection alone need not implement maximal munch with bounded memory; the upper theorem requires the supplied streaming presentation.

The nonempty vocabulary $\mathcal{V} = \{v_1, \ldots, v_M\} \subseteq \Sigma^+$ is a one-state detokenizing transducer, mapping identifier i to byte string v_i. Put $D = \sum_i |v_i|$ and $L = \max_i |v_i|$. At a token-boundary prefix $x \in \mathcal{V}^*$, define

$$\mathsf{mask}_x[i] = 1 \quad \Longleftrightarrow \quad \exists y \in \mathcal{V}^* : xv_iy \in K. \tag{1}$$

We call this problem tokenized prefix completability (TPC). The online interface preprocesses the static input, answers Mask, and performs $\mathsf{Commit}(i)$ by appending v_i. EOS is the separate bit $[x \in K]$. Failed prefixes have zero masks thereafter. Future output is constrained to $\mathcal{V}^*$: for $K = \{\mathsf{ab}\}$ and $\mathcal{V} = \{\mathsf{a}\}$, byte-prefix liveness and tokenized completion differ.

The static size g counts grammar symbols, lexer transitions, and any supplied normalized presentation in words. A compiler's cost and output size are included when a presentation is constructed from G. The horizon $n \geq 2$ counts committed bytes; t token commits satisfy $t \leq n \leq tL$. We use a word-RAM with $w = \Theta(\log n)$ and require all static and dynamic storage to be addressable. An explicit mask occupies $B = \lceil M/w \rceil$ words and must be written into caller-owned storage; returning a pointer to a cached row is a different interface. Sampling is external to both operations.

For the deterministic upper bound, P has controls Q, stack alphabet Γ, elementary pushes/pops/control changes, and explicit EOF processing. Acceptance is by final control or empty stack; any supplied regular acceptance automaton is included in g. It is boundary-terminating: each next byte is consumed or rejected after finitely many operations. The current boundary is immediately after consuming the previous byte,

before any optional silent closure. Let r_i be the elementary commit work, including silent transitions, and $W_t = \sum_{i \le t} r_i$. A c_0-bounded-work presentation has $r_i \le c_0(|v_i| + 1)$; an a-linear-work presentation has $W_t \le a(n + 1)$. These are execution hypotheses. Deterministic visibly pushdown presentations satisfy the first with a constant; an LR reduction burst is charged to r_i.

Theorem 1 (Regular-family lower bounds). *Under OMv, for every $\epsilon > 0$ there is no uniform exact engine with $\mathrm{poly}(g + D)$ preprocessing and amortized $O(n^{1/2-\epsilon})$ time per step on all right-linear grammars. This holds with an identity lexer, $M = D = 6$, and $L = 1$, on a family with $g = \Theta(m^2)$ and $n = \Theta(m^2)$. The same family admits $O(1 + m/w)$ worst-case steps and $O(m^2)$ preprocessing and space.*

With one polynomial preprocessing and m independently initialized streams of horizon $n = \Theta(m)$, constant-time initialization, and $g = \Theta(n^2)$, the excluded bound is instead amortized $O(n^{1-\epsilon})$ per step.

Here and below a step includes the query and selected commit. The lower bound excludes a fixed polynomial saving, allows logarithmic improvements, and has no hidden online factor polynomial in g.

Theorem 2 (Explicit stack index). *For a supplied deterministic presentation, a joint bottom-to-top configuration classifier with*

$$s = O\big(g(M + 1)(D + L + 1)\big)$$

NFA states can be constructed in $\mathrm{poly}(g + D + L)$ time. Determinizing it yields a VIABLE index with preprocessing $\mathrm{poly}(g + D + L) + 2^{O(s)}$ and static space

$$O\big(\mathrm{poly}(g + D + L) + 2^s(|\Gamma| + |Q|B)\big).$$

On addressable instances, mask queries cost $O(B)$ worst-case, commits cost $O(|v_i| + r_i + 1)$, and dynamic space is $O(1 + W_t)$ words. Thus bounded-work presentations have $O(B + c_0L + c_0)$ worst-case steps, and linear-work presentations have total online time $O(tB + an + a + t)$.

An explicit smaller DFA classifier can replace the 2^s subset bound. The theorem measures the supplied pushdown representation, including its lexer; it is not a polynomial compilation theorem for arbitrary CFGs.

Theorem 3 (Block-tabulation tradeoff). *Given the classifier of Theorem 2, choose $1 \le b \le \min(s, w)$, let $k = \lceil s/b \rceil$ and $d = \lceil s/w \rceil$. Besides polynomial classifier construction, preprocessing time and static space are*

$$O\big(\mathrm{poly}(s + g + D) + 2^b k(|\Gamma|d + |Q|B)\big). \tag{2}$$

Mask queries cost $O(kB)$, a push costs $O(kd)$, and a pop costs $O(1)$. Committing token i costs $O(|v_i|+(r_i+1)kd)$, with dynamic space $O((1 + W_t)d)$. All bounds are worst-case for the specified operations.

Choosing $b = \Theta(\log s)$, capped by w, gives polynomial tabulation size; the full subset index gives output-optimal queries at exponential static cost. This is an explicit implementation tradeoff, without an assertion that it is optimal among all index structures.

For the DFA row of Table 1, precompute every token transition and mark states reaching acceptance through vocabulary-token edges. Both tasks take $O(hD)$ time; cached rows and token transitions give the stated online bound.

Table 1. Common-parameter comparison. $B = \lceil M/w \rceil$; n counts bytes in the specified stream. The matrix-family lower bounds and its upper bound match in polynomial exponent.

Representation	Preprocessing	Online guarantee
Supplied DFA, h states	$O(hD)$	$O(B + L)$ per step
PDA, subset index	$\mathrm{poly}(g + D + L) + 2^{O(s)}$	$O(B)$ query; $O(L + r_i)$ commit
PDA, b-bit blocks	Equation (2)	$O(kB)$ query; $O(L + (r_i + 1)kd)$ commit
Matrix grammar family	$O(g)$	$O(1 + \sqrt{g}/w)$ per step
Same family, one stream	$\mathrm{poly}(g + D)$	OMv excludes $O(n^{1/2-\epsilon})$; $g = \Theta(n)$
Same family, independent streams	$\mathrm{poly}(g + D)$	OMv excludes $O(n^{1-\epsilon})$; $g = \Theta(n^2)$

3 Configuration Compilation and Maintenance

The technical overview is a three-stage construction: build regular future tests, reverse them into a joint stack classifier, and maintain its annotation at persistent stack nodes. We give the construction and its size accounting together.

Lemma 4 (Regular constrained futures). *For a normalized pushdown machine and a regular input language R, the configurations that can consume a word in R and accept after EOF form an effectively constructible regular set.*

Proof. Take the product with an automaton for R, updating its state on consuming transitions. Permit EOF only at an accepting R state. A regular target after EOF is represented by an NFA reading stacks top to bottom, with product controls as initial states and separate internal target states. Initially no edge enters a control state. For each pushdown rule $(p, A) \longrightarrow (q, \beta)$, $|\beta| \leq 2$, add $p \xrightarrow{A} u$ whenever a current path $q \xrightarrow{\beta} u$ exists. This saturation adds edges but no states, and terminates in polynomial time [3].

For soundness, rank added edges by insertion time. If an accepting path starts with a target edge, it stays inside the original target automaton. Otherwise replace its first edge by its witnessing rule and the earlier path for β. This takes one pushdown step and replaces one rank by smaller ranks. Well-founded multiset order implies that expansion ends in an original target path. For completeness, induct on a run to the target; split the accepting path for βu after β and use the edge added for the first rule. Selecting controls with the initial R state yields the asserted source configurations. □

Compilation in Theorem 2. For each vocabulary token z, use $R = z\mathcal{V}^*$ in Lemma 4. A vocabulary trie with returns from token ends recognizes $\mathcal{V}^*$ using $O(D + 1)$ NFA states; prepending z uses $O(D + L + 1)$ states. Eliminate input-automaton epsilon edges without adding states. Its product with the explicit presentation and regular accepting target has $O(g(D + L + 1))$ saturation states. Add one analogous test for EOS. Saturation adds no states and has polynomially many possible edges. Across $M + 1$ tests their total size is the stated s.

Reverse these stack NFAs and take their disjoint union, using all reversed accepting states as initial states. Each accepting state is labeled by the original control q and candidate identifier i. On a bottom-to-top stack u, its active set $S(u)$ therefore determines the exact mask: bit i is set when a state labeled (q, i) is active. Multiple labels are combined by OR. The additional EOS label changes the packed output size by at

Algorithm 1 VIABLE with a determinized stack classifier

Maintain parser control q and stack root u storing index state $H(u)$.
function MASK(out)
    Copy B words from $Rows[q, H(u)]$ to out.
end function
function PUSH(A)
    $u \leftarrow Node(A, u, \delta_H(H(u), A))$.
end function
function POP
    $u \leftarrow parent(u)$.
end function
function COMMIT(i)
    Run the presentation on v_i, maintaining annotations on every push/pop.
    Charge consuming and silent work to $|v_i| + r_i$.
end function

most one word. Vocabulary continuations are included in R, so the construction works without assuming byte-complete vocabularies.

The subset DFA has at most 2^s states. Compute its transitions and, for each control and subset, its packed mask row. Naive finite-set operations give $2^{O(s)}$ time and the stated transition/row space. The separate polynomial term contains the input classifier and its construction. For a fixed alphabet, $M \leq \sum_{\ell \leq L} |\Sigma|^\ell$ and $D \leq LM$ make the dependence on grammar size and maximum token length elementary as well. □

MAINTENANCE IN THEOREM 2. Store the subset-DFA state for the complete bottom-to-node stack at each persistent node. A push computes one DFA transition and records its parent; a pop restores the parent state. Induction on operations establishes the annotation invariant. Control changes select a different output row. Addressability ensures each state identifier and pointer fits in a word. A query copies B words; committing executes the deterministic parser and pays constant index overhead per elementary operation. At most one node is allocated per push. Failed commits enter a zero-row dead state. Summing gives the stated work and space bounds and the two execution corollaries. □

PROOF OF THEOREM 3. Retain the active set of the joint reversed NFA as d packed words, partitioned into k groups of at most b states. For each stack symbol, group, and one of its 2^b bit patterns, tabulate the OR of the successor-state bitsets of the selected states. Each entry has d words. For each control, group, and pattern, also tabulate the OR of the selected states' output labels as a B-word mask. The tables contain at most the number of words in (2). Recurrence on the least significant selected bit fills them in the same asymptotic time after polynomial setup.

Because $b \leq w$, extracting a group takes constant word operations even across a word boundary. A push looks up k transition entries and ORs them into a new d-word annotation, costing $O(kd)$; correctness is distributivity of NFA transition over union. A query similarly ORs k output entries in $O(kB)$. Store annotations with persistent parents; popping changes one pointer. Control changes affect only which output table is used. The allocation and commit bounds follow by charging all pushes, pops, and control changes to r_i, with at most d words per allocated node. □

A regular stack property can depend on its whole depth, such as parity of one symbol; its finite-state annotation need not be a bounded suffix. Conversely, persistence does not remove parsing work. In $S \to Tc$, $T \to aT \mid b$, a conventional LR parser can perform $\Theta(d)$ reductions on lookahead c after reading $a^d b$. The index gives its mask without speculative reductions, while committing c charges the cascade. This language is regular, so the example is about the parser execution rather than a language lower bound.

4 The Regular OMv Construction

For an $m \times m$ Boolean matrix A, use six one-byte tokens $\{0, 1, \#, a, !, ?\}$ and define

$$B_m = \{0,1\}^m \# a^m !, \tag{3}$$

$$T_A = \{v \# a^i ? : v \in \{0,1\}^m,\ 1 \le i \le m,\ (Av)_i = 1\}, \tag{4}$$

$$K_A = B_m^*(\{\varepsilon\} \cup T_A). \tag{5}$$

The universal branch B_m^* maintains liveness independently of row answers.

Lemma 5 (Grammar size and query predicate). *A right-linear grammar of size $O(m^2)$ for K_A can be built in $O(m^2)$ time and padded to $\Theta(m^2)$. For $p \in B_m^*$,*

$$\mathsf{mask}_{pv\#a^i}[?] = (Av)_i \qquad (1 \le i \le m). \tag{6}$$

Proof. Build an NFA. A boundary state accepts, or starts a universal block, counting m bits, #, m copies of a, and ! before returning to the boundary. This costs $O(m)$ states.

The test branch counts bit positions until selecting a 1 at position j. States (j, k) count the remaining $m - j$ bits while retaining j; after #, states (j, i) count copies of a. An outgoing ? to a terminal accepting sink exists exactly for $1 \le i \le m$ with $A_{ij} = 1$. These states and edges number $O(m^2)$. Starting alternatives use same-letter edges, so no extra epsilon structure is needed. Convert each edge $q \xrightarrow{c} r$ to $N_q \to cN_r$ and each final state to $N_q \to \varepsilon$.

After p, the current block contains no !. The universal branch cannot read ?; the test branch can read it exactly when some selected j has both $v_j = 1$ and $A_{ij} = 1$. It then accepts immediately, and its sink has no outgoing edges. No suffix repairs a missing witness. This proves (6); redundant states give the uniform padded size. □

Lower bounds in Theorem 1. Use the standard polynomial-preprocessing consequence of OMv [10]; its precise form is Theorem 2.2 of the full version, arXiv:1511.06773. Preprocess the grammar. On each revealed vector v, commit $v\#$, then one a per row. After the ith a, output the mask's ? bit as $(Av)_i$. Commit ! after the last row. Only then request the next vector. The stream never commits ? and every actual prefix can extend to a universal block, including for an all-zero matrix.

For m vectors, the trace has $n = m(2m + 2) = \Theta(m^2)$ bytes and $O(m^2)$ calls. An amortized $O(n^{1/2-\epsilon})$ step bound gives online time $O(m^{3-2\epsilon} + m^2)$, contradicting OMv with polynomial matrix preprocessing. Grammar construction and preprocessing fit that budget. Constant M absorbs any polynomial vocabulary factor.

For independent streams, initialize a fresh stream for each vector after the same static preprocessing. Each horizon is $n = 2m + 2 = \Theta(m)$; m streams still use $O(m^2)$ calls. An $O(n^{1-\epsilon})$ step bound gives

Table 2. Actual deterministic validation coverage. Depth tests use eight specified depths; the interval does not denote exhaustive depth coverage. No timing values are inferred.

Check	Count
Dyck vocabulary / maximum length	340 / 4
All byte prefixes of length at most 6	5,461
Exhaustively compared mask bits	1,856,740
Additional depth-test mask bits	2,720
Matrices of orders 1 through 3	530
Exact matrix-row probes	12,420
Universal-prefix liveness checks	70,500
False-positive / false-negative bits	0 / 0
Stack depths in operation test	0–4,096
64-bit row words copied at each depth	6

$O(m^{3-\epsilon} + m^2)$ online time, the same contradiction. Initialization is charged explicitly. The different exponents follow from the two different byte horizons. □

Family upper bound in Theorem 1. Store packed matrix rows, the current vector, and counters for the block shape. Before #, the universal branch determines the legal bits. After i copies of a, it determines the ordinary next byte: a for $i < m$, or ! for $i = m$. For $1 \le i \le m$, determine the ? bit by ANDing packed row i with the vector and testing for a nonzero word in $O(1 + m/w)$ time; before the first a, that bit is zero. A valid committed ? reaches the terminal accepting state; invalid commits enter a dead state. A valid ! clears the vector in $O(1 + m/w)$ time and returns to the boundary. The counters also determine EOS. This covers every input, with $O(m^2)$ preprocessing and space. On the lower-bound trace $m = \Theta(\sqrt{n})$. □

5 Sanity Checks and Reproducibility

The accompanying artifact supplies a Rust Dyck-mask specialization, Python binding, independent stack oracle, and an independently executed matrix NFA. Its deterministic checks regenerate Table 2. For the two-bracket Dyck language, the vocabulary contains every string of lengths one through four over four bracket bytes. Exact liveness is checked by reading the entire candidate and rejecting any mismatched close or underflow; every surviving stack has a closing completion. The implementation precomputes 31 top-suffix classes with 340 mask bits each and stores six 64-bit words per row: 186 words, or 1,488 bytes of valid-row mask payload. The Rust allocation includes an unused zero row, giving 192 words and 1,536 bytes of row payload, excluding allocator and other data.

Depth checks use exactly $\{0, 1, 4, 16, 64, 256, 1024, 4096\}$. Matrix checks enumerate every matrix of orders one through three, every vector, and every row. Each liveness check examines a prefix of *two consecutive identical complete universal blocks*, including the empty prefix. Counts include repeated strings across instances; they are executed checks, not distinct-language cardinalities.

Artifact access. The source, commands, and raw results are provided as an embedded artifact package. Its README builds the shared library with `rustc` and runs `python3 verify.py`. The package also includes generic NFA block-tabulation checks and its explicit logical-payload accounting. These are finite

correctness and operation-count checks. Their scope is the implemented Dyck specialization, small NFAs, and reduction instances; production lexer compilation, general LR execution, and model-serving latency are separate empirical questions.

6 Scope and Semantic Boundary

The regular family gives matching polynomial exponents under a polynomial preprocessing budget. The pushdown index gives explicit compiler/query/update tradeoffs for a supplied deterministic presentation. Its exponential full-index endpoint and the lower bound's succinct grammar input are different representations and budgets. These statements support a representation-sensitive account of decoding rather than a determinism-only dichotomy. General LR(1) worst-case commit acceleration and optimal intermediate index tradeoffs remain research questions.

Typed completion introduces an additional boundary. Goal-directed simply typed normal-term completion is PSPACE-complete by classical inhabitation complexity; we state and prove the precise corollary in Appendix A. Universal divergent inhabitants fill independently typed holes when divergence is allowed, but completed ill-typed subterms remain obstructions. Semantic constraints therefore need their own completion model.

AI methodology. OpenAI Codex materially assisted the formalization, candidate proofs and tradeoff construction (Sections 2–4), implementation and finite checks (Section 5), literature work, and drafting throughout. The disclosed verification consists of executable finite comparisons and the written arguments; it is not machine-checked proof or independent certification of novelty. Human authors retain responsibility for the submitted content.

A Typed Completion as an Auxiliary Corollary

Types are atoms and arrows. The context Δ and target A are supplied externally. At goal $B \to C$, the long normal syntax is $\mathtt{l}(e)$, introducing a variable of type B and solving C. At atomic goal p, choose an in-scope variable $x : B_1 \to \cdots \to B_k \to p$ and write $\mathsf{h}\langle j\rangle(e_1, \ldots, e_k)$, where j is its de Bruijn index in terminated binary without leading zeros. Each e_i has goal B_i, and parentheses remain for $k = 0$. This is the goal-directed serialization used below.

Corollary 6 (Normal typed-prefix completion). *Given Δ, A and a text prefix in this serialization, extendibility is PSPACE-complete in their combined size. Empty-prefix completion in unrestricted STLC source syntax is PSPACE-hard.*

Proof. Let N be the total input size. Scan the prefix with frames storing expected goal, scope, delimiters, and the remaining argument positions of the selected head. An arrow goal determines a lambda's domain; a completed head determines its result atom and argument types. Reject an unknown index, wrong result atom, or wrong delimiter. A partial binary index has at most N in-scope completions, which can be enumerated sequentially.

There are at most N explicitly opened frames. Their types are pointers to subformulas of the supplied types, each of size at most N; at most $O(N^2)$ remaining argument obligations can be enumerated. Each context contains the original assumptions and at most N explicitly introduced variables. Even materializing

all contexts and goals takes polynomial space. The unobserved portion of each argument is an independent inhabitation obligation: ordinary STLC permits weakening and contraction, so filling one argument consumes no resource needed by another. A completion supplies inhabitants for all obligations; conversely inhabitants, restored delimiters, and a compatible head choice give a completion.

Invoke the polynomial-space simple-type inhabitation procedure [18, 19] separately for each obligation, reusing its workspace, and test the finitely many partial-head alternatives sequentially. Each procedure receives polynomial-size input; the outer enumeration and contexts occupy polynomial space. This proves membership. For hardness, a closed type A maps to empty context and empty prefix at target A. It is completable exactly when A is inhabited, since every inhabited simple type has a long normal inhabitant. The same empty-prefix reduction gives unrestricted-source hardness. □

If every type has a permitted term Ω_A, substitution fills every hole in a well-typed skeleton with independent typed holes. General recursion can supply fix $x{:}A.x$ when divergence is allowed. In contrast, if $c : o$, a prefix fixing a fully delimited application $c\ c$ cannot be completed to a well-typed program: application requires an arrow type for the first c. This obstruction persists in the presence of other holes or divergent inhabitants.

References

[1] Amir Abboud, Arturs Backurs, and Virginia Vassilevska Williams. 2015. If the Current Clique Algorithms are Optimal, So is Valiant's Parser. In *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE, 98–117. doi:10.1109/focs.2015.16

[2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2024. Guiding LLMs The Right Way: Fast, Non-Invasive Constrained Generation. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 3658–3673. https://proceedings.mlr.press/v235/beurer-kellner24a.html

[3] Ahmed Bouajjani, Javier Esparza, and Oded Maler. 1997. Reachability analysis of pushdown automata: Application to model-checking. In *CONCUR '97: Concurrency Theory*, Antoni Mazurkiewicz and Józef Winkowski (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 135–150.

[4] Karl Bringmann, Allan Grønlund, Marvin Künnemann, and Kasper Green Larsen. 2024. The NFA Acceptance Hypothesis: Non-Combinatorial and Dynamic Lower Bounds. *TheoretiCS* Volume 3, Article 22 (Oct 2024). doi:10.46298/theoretics.24.22

[5] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ziyi Xu, Yilong Zhao, Ruihang Lai, and Tianqi Chen. 2025. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. In *Proceedings of Machine Learning and Systems*, M. Zaharia, G. Joshi, and Y. Lin (Eds.), Vol. 7. MLSys. https://proceedings.mlsys.org/paper_files/paper/2025/file/5c20ca4b0b20b0bd2f1d839dc605e70f-Paper-Conference.pdf

[6] Bartłomiej Dudek and Paweł Gawrychowski. 2024. Online Context-Free Recognition in OMv Time. In *35th Annual Symposium on Combinatorial Pattern Matching (CPM 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 296)*, Shunsuke Inenaga and Simon J. Puglisi (Eds.). Schloss

Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 13:1–13:9. doi:10.4230/LIPIcs.CPM.2024.13

[7] Hervé Gallaire. 1969. Recognition time of context-free languages by on-line Turing machines. *Information and Control* 15, 3 (September 1969), 288–295. doi:10.1016/s0019-9958(69)90463-x

[8] Moses Ganardi, Louis Jachiet, Markus Lohrey, and Thomas Schwentick. 2022. Low-Latency Sliding Window Algorithms for Formal Languages. In *42nd IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2022) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 250)*, Anuj Dawar and Venkatesan Guruswami (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 38:1–38:23. doi:10.4230/LIPIcs.FSTTCS.2022.38

[9] Daniel Gibney and Sharma V. Thankachan. 2021. Text Indexing for Regular Expression Matching. *Algorithms* 14, 5 (April 2021), 133. doi:10.3390/a14050133

[10] Monika Henzinger, Sebastian Krinninger, Danupon Nanongkai, and Thatchaphol Saranurak. 2015. Unifying and Strengthening Hardness for Dynamic Problems via the Online Matrix-Vector Multiplication Conjecture. In *Proceedings of the forty-seventh annual ACM symposium on Theory of Computing (STOC '15)*. ACM, 21–30. doi:10.1145/2746539.2746609

[11] Hantao Hua, Jiming Su, Hao Tang, Yiping Yao, and Feng Zhu. 2026. Gram2Token: Enabling Run-time GPU-Native Grammar-Constrained Decoding for LLMs. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*, Tong Zhang, Miroslav Dudik, Martin Jaggi, Alekh Agarwal, Sharon Li, Dale Schuurmans, Jerry Zhu, Felix Berkenkamp, Hanze Dong, and Alberto Bietti (Eds.). PMLR, 46169–46185. https://proceedings.mlr.press/v306/hua26b.html

[12] Lillian Lee. 2002. Fast context-free grammar parsing requires fast Boolean matrix multiplication. *J. ACM* 49, 1 (2002), 1–15.

[13] Yongmin Li, Yihong Dong, Jia Li, and Ge Li. 2026. Efficient Grammar-Constrained Decoding via Parser Stack Classification. arXiv:2608.03065 [cs.SE] https://arxiv.org/abs/2608.03065

[14] llguidance contributors. 2026. llguidance: Low-level Guidance. https://github.com/guidance-ai/llguidance Software documentation, accessed October 31, 2026.

[15] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. 2025. Type-Constrained Code Generation with Language Models. *Proceedings of the ACM on Programming Languages* 9, PLDI (June 2025), 601–626. doi:10.1145/3729274

[16] Shaan Nagy, Timothy Zhou, Nadia Polikarpova, and Loris D'Antoni. 2026. ChopChop: A Programmable Framework for Semantically Constraining the Output of Language Models. *Proceedings of the ACM on Programming Languages* 10, POPL (Jan. 2026), 1905–1932. doi:10.1145/3776708

[17] Kanghee Park, Timothy Zhou, and Loris D'Antoni. 2025. Flexible and Efficient Grammar-Constrained Decoding. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.). PMLR, 48262–48275. https://proceedings.mlr.press/v267/park25l.html

[18] Richard Statman. 1979. Intuitionistic propositional logic is polynomial-space complete. *Theoretical Computer Science* 9, 1 (July 1979), 67–72. doi:10.1016/0304-3975(79)90006-9

[19] Vítězslav Švejdar. 2003. On the polynomial-space completeness of intuitionistic propositional logic. *Archive for Mathematical Logic* 42, 7 (01 Oct 2003), 711–716. doi:10.1007/s00153-003-0179-x

[20] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2025. SynCode: LLM Generation with Grammar Augmentation. *Transactions on Machine Learning Research* (2025). https://openreview.net/forum?id=HiUZtgAPoH

[21] Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. arXiv:2307.09702 [cs.CL] https://arxiv.org/abs/2307.09702